VisionGuard AI

Technical Report

How VisionGuard turns ordinary camera feeds into live detections, events and alerts: the problem, the users, the architecture, OpenCV 5, AWS deployment, evaluation, limitations and responsible use.

Repository	https://github.com/RIT8603/VisionGuard
Demo video	https://youtu.be/Mmx9-0xzOfI
Hosted portal	https://www.vision-guard.tech
Team	Ritesh (Computer Vision & Backend) and Vanessa Victorino (UI & UX)
Date	27 October 2026

Summary

VisionGuard AI is a configurable video-analytics platform. A camera feeds an OpenCV 5 Vision Core; the core runs only the AI modules a site has switched on, from people detection and tracking to restricted zones, fall detection, PPE, number plates and face recognition. Modules publish structured results and events, which an operator sees in a web portal as annotated live video, an alert feed and reports.

The design goal is that one platform covers many site needs without new code: moving from a laptop webcam to a CCTV camera is a configuration change, and adding a capability is one module file plus one registry line.

At a glance

Area	Facts
Vision core	Python 3.12 and OpenCV 5 (`opencv-python==5.0.0.93`)
AI modules	Pluggable detection, tracking, recognition and analysis modules; enabled per camera, switchable while running
Models	YOLO11 (OpenVINO by default, PyTorch optional), YOLOv8n-face, OpenCV SFace, pose, OpenVINO Re-ID and plate models
Interfaces	FastAPI backend (port 8080): MJPEG stream, events, alerts, `/api/face/*`, `/api/reports/*`; React 19 / TypeScript / Vite portal
Storage	SQLite for face templates and events; snapshots on disk under `data/` (excluded from Git)
Deployment	Local PC, or AWS: Nginx frontend EC2 plus a CPU or GPU backend EC2
Tests	Automated suite: 1345 passed, 1 failed (known config expectation), 1 skipped

1. Problem

Cameras are cheap and everywhere; watching them is not. Most CCTV footage is never seen live. It is searched after something has already gone wrong, and one person cannot watch a wall of screens attentively for a whole shift. When sites do buy video analytics it usually comes as separate single-purpose products (people counting, number-plate reading, face access control), each with its own hardware, licence and console, and often tied to one camera vendor.

A small factory, warehouse, office or campus typically needs a handful of these capabilities, not all of them, and needs them on the cameras it already owns.

What VisionGuard does about it

- **One platform, many modules.** Each camera runs only the modules it needs. Operators switch modules on and off from the portal without restarting the pipeline.
- **Works with existing cameras.** Webcam, video file, RTSP / IP camera and YouTube stream sources share one interface; changing source is a configuration change.
- **Events, not just boxes.** Rule modules (zones, loitering, tailgating, falls, abandoned objects, PPE) turn detections into alerts that an operator can acknowledge.
- **Ordinary hardware.** Runs on a CPU (OpenVINO speeds up Intel CPUs) and uses an NVIDIA GPU when present.

2. Users

The intended users below follow from the module set. This report does not include results from a customer site.

User	What they need	Modules they use
Security / control-room operator	To be told when something needs attention, instead of watching every screen.	Restricted zones, after-hours intrusion, loitering, tailgating, weapon detection, unknown-person alerts
Health and safety officer (factory, warehouse, site)	To spot unsafe situations quickly.	PPE detection, fall detection, crowd counting and alerts, abandoned object
Facility or HR administrator	To know who is on site and when.	Face Registry, automated attendance, entry / exit gate counter, Reports with CSV export
Site or car-park manager	To follow vehicles and stock.	Vehicle monitoring, number-plate recognition (ANPR), inventory monitoring
Integrator / developer	To add a capability for one customer without touching the rest.	Module interface: one file plus one registry line; YAML configuration

Operators use the web portal: Dashboard, Live Monitoring, Cameras, Events and Alerts, Modules, Face Registry, Reports and Settings.

3. Architecture

Each camera runs its own pipeline. With several cameras, every pipeline runs in a separate worker process, while the dashboard API, alert centre and Face Registry stay in the main process.

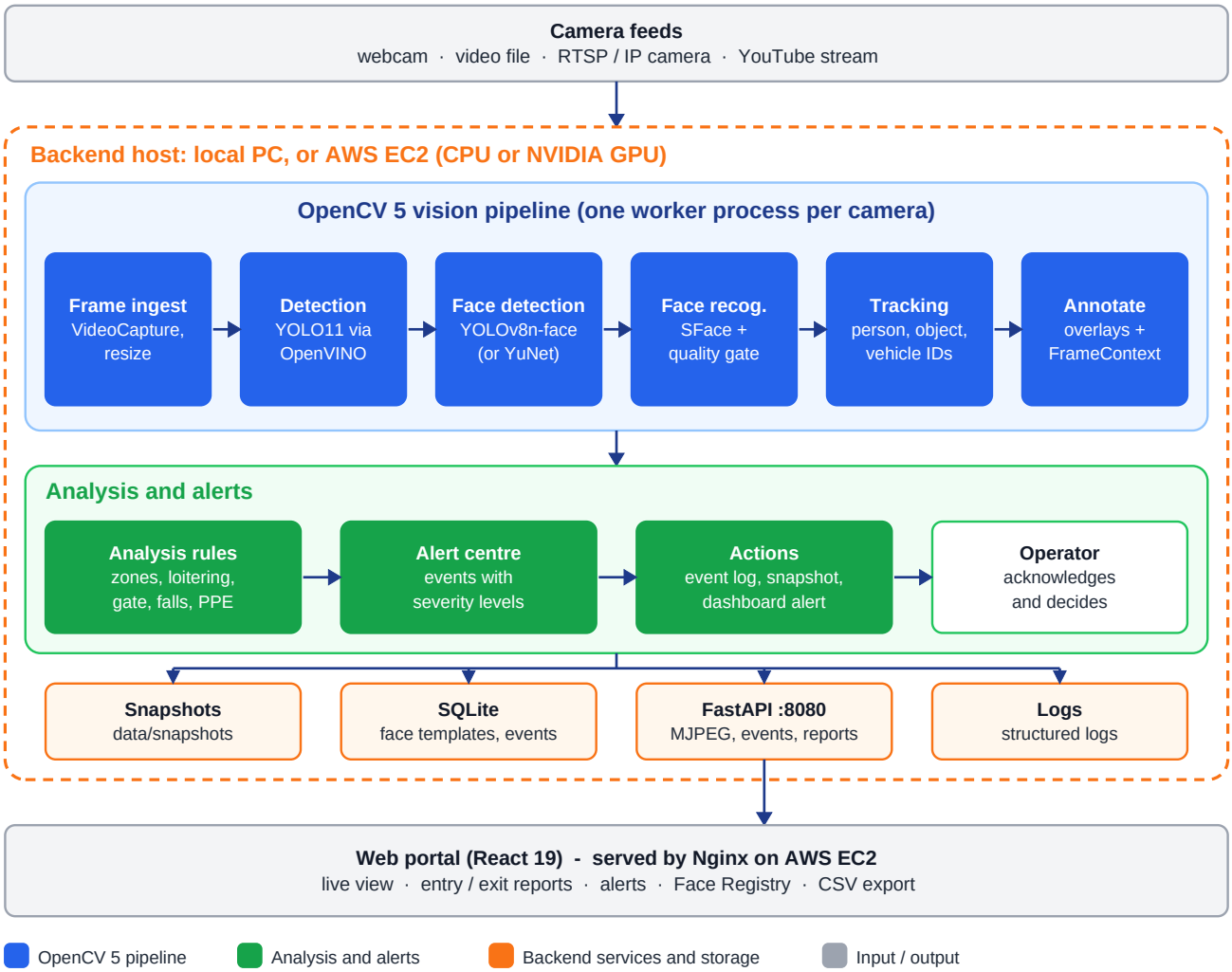

Figure 1. System architecture, simplified: each stage runs only when its module is enabled, in dependency order. The backend runs locally today; the AWS backend host is prepared but not yet verified on EC2 (section 5).

How a frame flows

1. A video source (`BaseVideoSource`) delivers a frame with a timestamp and source id. RTSP adds reconnect, open / read timeouts and stale-frame dropping.
2. The Vision Core resizes the frame, measures FPS and latency, and builds a `FrameContext`.
3. The module registry runs enabled modules in dependency order: detection, then tracking, then analysis. Disabled modules are never initialised, and circular dependencies are not allowed.
4. Modules write detections, tracks and events into the FrameContext and draw their overlays.
5. The worker sends the annotated frame, events and status to the main process over a queue.
6. FastAPI streams MJPEG to the portal, records events, raises alerts and serves the Face Registry and Reports APIs.

Design decisions

Decision	Why
Module registry with declared dependencies	A site runs only what it needs. Adding a module is one file plus one registry line.
Detection separated from decision	Detectors report what they see; analysis modules decide what it means ("a helmet is here" vs "this worker is in violation"). False-alarm logic stays in one testable place.
`DetectorBackend` interface	Modules never import a neural-network library. OpenVINO backend: 243 MB, 1.1 s start-up; Ultralytics backend: 923 MB, 4.3 s. ONNX Runtime or OpenCV DNN can be added without changing any module.
Shared inference	Detectors on the same weights share one inference per frame: 9.0 to 19.3 FPS on CPU, with byte-identical results.
One process per camera	Python runs one thread at a time. Separate processes used every core: 4 cameras went from 9.6 to 23.6 FPS. Cost: about 1-1.5 GB RAM per camera.
Motion gate before face detection	The face detector is skipped when under 0.2% of pixels changed, and the last faces are reused.

4. OpenCV 5

OpenCV 5 is the Vision Core. `requirements.txt` pins `opencv-python==5.0.0.93`, and every frame passes through OpenCV from capture to the JPEG sent to the browser. Neural networks do the object detection; OpenCV does everything around them, and runs face recognition itself.

Stage	OpenCV 5 API	Used for
Capture	`VideoCapture` with `CAP_PROP_OPEN_TIMEOUT_MSEC`, `READ_TIMEOUT_MSEC`, `BUFFERSIZE=1`	Webcam, video file, RTSP and YouTube sources. Timeouts make a stalled camera fail fast; a one-frame buffer drops stale frames.
Pre-processing	`resize` (INTER_AREA), `cvtColor`	Shrinking frames for detection and for the live stream.
Face recognition	`FaceRecognizerSF` (SFace)	Aligns each face and turns it into a numeric template, matched against the Face Registry.
Face detection (option)	`FaceDetectorYN` (YuNet)	CPU face detector with 5 landmarks. The shipped config uses YOLOv8n-face, which found more small CCTV faces; YuNet is one setting away.
Face quality gate	`Laplacian` variance	Blurred face crops are rejected at enrolment and before recognition.
Low light, hazy faces	`createCLAHE` on the LAB L channel, `LUT` gamma	Low-light module and contrast boost for small CCTV head crops.
Detection post-processing	`dnn.NMSBoxesBatched`	Per-class non-maximum suppression in the OpenVINO backend, so it needs no PyTorch.
Zones and lines	`pointPolygonTest`, `fillPoly`, `polylines`	Restricted zones, gate lines and their overlays.
Motion gate	`absdiff` on a blurred greyscale copy	Skips face detection on still frames.
Output	`rectangle`, `putText`, `addWeighted`, `imencode`, `VideoWriter`	Annotated overlays, the MJPEG live stream and recordings.

OpenCV 5's Python wheel no longer bundles face model data, so the YuNet and SFace ONNX files are fetched from the OpenCV Zoo with `python -m tools.download_face_models`. A module enabled without its model fails at start-up with the download URL, rather than silently finding nothing.

5. AWS deployment

On AWS VisionGuard uses two EC2 instances: a small frontend that serves the portal and a backend that runs the vision pipeline. The backend can start on CPU and later move to an NVIDIA GPU instance; CPU and GPU are alternative backend hosts, not two halves of one pipeline. Scripts and the full guide are in `deploy/` (`DEPLOY.md`).

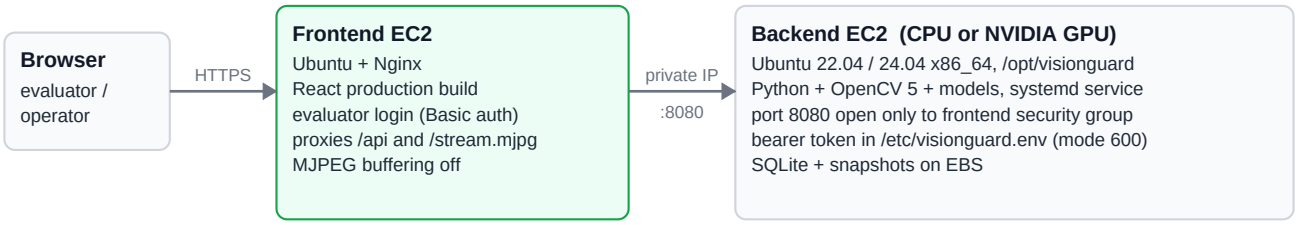

Figure 2. AWS layout. The backend port is never opened to the internet.

Concern	How it is handled
Network	Both instances in one VPC. Backend port 8080 accepts traffic only from the frontend's security group; SSH only from the administrator's IP.
Secrets	Nginx checks the evaluator login, then replaces it with a separate backend bearer token upstream. The token stays in `/etc/visionguard.env` (root, mode 600): never in frontend code, a URL or Git.
Service	`visionguard-backend` systemd unit; starts on boot. Re-running setup keeps module / camera choices and the token.
GPU switch	Same scripts with `gpu` mode: CUDA 13.0 PyTorch wheels, a real CUDA tensor check, then a documented state migration (stop service, archive SQLite and snapshots, restore).
Cost	Stop the backend EC2 after a demo. Persistent data lives on EBS.

Status on 27 October 2026

- The portal is live at https://www.vision-guard.tech, served over HTTPS by Nginx on an AWS-hosted Ubuntu instance. It is the interface only: no backend is attached at the time of writing, so its API calls return 502 and live video needs a running backend. The demo video shows the full system working.
- Deployment scripts were validated locally: 311 deployment, configuration, API and hardening tests passed (1 skipped); real factory CCTV ran through the generated CPU and CUDA profiles for 40 + 40 frames with 0 failed frames; Nginx login, token forwarding, same-origin checks and five simultaneous MJPEG streams passed.
- Not yet verified on EC2: the backend install, behaviour after a reboot, private networking between the instances and GPU inference on AWS.

6. Evaluation

Method. `tools/benchmark_cv.py` runs the real registry, models, pipeline and alert centre on real video; no fake detector is used. Ground truth is labelled by hand on raw frames, never copied from model output. Detections are matched by class with Hungarian matching at IoU >= 0.5 to give precision and recall. Where no annotation exists the result is reported as 'not measured', never as a made-up score. Measurements were taken on 26 October 2026; the protocol is in `docs/CV_BENCHMARK.md`.

Module results

Module	Measured result	Limit
Face recognition	14 identities, 193 templates, 414 labelled photos: **384 correct**, 29 rejected as unknown, 1 wrong name.	Enrolment photos, not independent CCTV.
PPE (helmet / vest)	Factory frame: 4 of 7 visible vests found, 0 false positives. Staged clip: 2 violation events, 2 alerts.	One annotated frame. Distant gear missed.
Person tracking	10 people in 2 labelled frames: 8 found, 0 false, 2 missed; **0 ID switches**.	Only 2 sampled frames.
Fall detection	1 fall event detected on a real clip; no false recovery event.	One staged event.
Abandoned object	1 abandoned-bag event (20 s threshold).	Heuristic; does not prove ownership.
Gate / tailgating	4 gate entries, 4 line crossings, 3 tailgating events for one four-person group.	One group.
Loitering	5 loitering alerts on a factory clip.	5 s threshold, not the 30 s default.
Person Re-ID, one camera	Right identity first for 68% of 62 people; at threshold 0.6, 77% re-identified, about 1% wrong matches.	Single camera footage.
Person Re-ID, two cameras	0 of 1 real two-camera pair matched at the default threshold.	Experimental; needs calibration.
OpenVINO vs PyTorch backend	1632 of 1632 boxes identical over 90 frames of 3 clips.	Equivalence, not accuracy.

Speed

Test	Result
4 cameras, one process each (RTX 5060)	Total **9.6 to 23.6 FPS**; GPU use 4-8% to 14-41%.
Shared model, 3 detectors (CPU)	**9.0 to 19.3 FPS**; frame latency 87.1 ms to 26.7 ms; results byte-identical.
7 modules at once, 640 px webcam (CPU)	20.0 FPS, 22.7 ms frame latency.
End-to-end FPS on test clips	Fall 26.9, abandoned object 28.2, gate / tailgating 25.0, PPE 14.6-17.3, vehicle / ANPR 4.8.

Clip FPS was measured with other camera workers running; it is not a hardware sizing promise.

Software checks

- Full automated suite: **1345 passed, 1 failed, 1 skipped**. The failure is a configuration expectation: the shipped config enables face detection while the test expects identity modules off.
- Real-model integration tests (detection, ANPR, Re-ID): 9 passed.

- Combined benchmark: 12 scenarios, 4,013 frames, 0 failed frames. Frontend type check and production build passed.

7. Limitations

Evaluation

- All results are small-sample checks on the team's own footage and public clips, not independent accuracy claims.
- Face accuracy was measured on enrolment photos, not on independent CCTV. Accuracy across demographic groups was not measured.

Models and modules

- Experimental: fall detection, PPE, abandoned object, face liveness and multi-camera Re-ID need camera-specific validation.
- Fire and smoke detection need custom model weights; they are not part of the default setup.
- ANPR reads only plates at least about 80 px wide; on a wide street view it boxes plates but reads none.
- There is no action recognition (fighting, theft). Intrusion covers only what place and time can decide.
- Low-light brightening did not help detection in our tests, so it is off by default.
- Time-based rules use capture time, so on a file played faster than real time their seconds do not match the video.

System

- Known open issues: stream slot accounting on overflow or rapid camera switching; face requests that can delay the API; daily gate totals above 1,000 events; a four-person crop limit in the long-gun model.
- Each camera worker uses about 1-1.5 GB of RAM. Alerts are held in memory and shown on the dashboard only; there are no SMS or e-mail notifications yet.
- The browser camera works only on localhost or HTTPS; elsewhere, enrol faces with Upload photos.
- On AWS, cameras on a local network need a VPN or another secured route to reach EC2.
- Ultralytics and its YOLO weights are AGPL-3.0, including when run through OpenVINO. A closed-source product would need non-AGPL weights and backend (for example ONNX Runtime).

8. Responsible use

VisionGuard can recognise people, so it processes personal and biometric data. The rules below are part of the product, not an afterthought; the separate Privacy and Consent Note gives the data inventory.

Intended use	Not intended for
Sites where people are told cameras are in use: workplaces, warehouses, campuses, car parks.	Covert surveillance, or filming people where they expect privacy.
Safety and security alerts that a person reviews and acts on.	Automatic disciplinary, legal or access decisions based on a face match alone.
Attendance for people who have agreed to be enrolled.	Identifying members of the public, or enrolling anyone without consent.

Privacy by design

- The Face Registry stores numeric face templates, not photographs. Deleting a user removes all of their templates.
- People who are not enrolled are shown as 'unknown'; they are never named.
- Identity processing (face detection, recognition, unknown-person alerts, attendance) is a separate set of modules that can be switched off on the Modules page. The shipped demo config enables face detection; a site that does not need identity should switch it off.
- Unknown-person and weapon snapshots are marked as personal data in the config and can be turned off with `save_snapshots: false`.
- The `data/` folder (faces, snapshots, events) is excluded from Git: the public repository holds no face database or enrolment photos.

Avoiding wrong names

A wrong name is worse than no name, so recognition is deliberately conservative: a face is named only above a similarity threshold and only when the best match clearly beats the runner-up; small, blurred or turned faces are never embedded; a name needs two agreeing matches before it is shown and is withdrawn after two disagreeing checks. If a body track overlapped someone else or was briefly lost, the name is held back as 'Checking...' until the face matches again. On enrolment photos the recogniser gave 1 wrong name in 414, with 29 left as unknown; that check did not include the confirmation rules above.

Human oversight and fairness

- Detectors report what they see; rules raise alerts; a person acknowledges and decides. VisionGuard does not act on its own.
- Face models (YOLOv8n-face, SFace) are public models. We have not audited their training data or measured accuracy across age, skin tone or gender, so results should be checked by a person before anyone acts on them.

Security

- The dashboard listens on localhost by default and checks every request's Host header. Binding to a network address requires a token.
- Write requests from another website are refused (cross-site check). Camera credentials are redacted in every log path.
- Ultralytics is pinned to an exact version because some past releases were published compromised.

Anyone can be removed on request: delete the user in the Face Registry and their files in `data/snapshots`.